

Checking Out the Prices: A Predictive Look Into the 2016/2017 Apartment Market in Queens

Final Project for Math 642, Data Science at Queens College

Due: May 26, 11:59PM by email

By Bryan Nevarez
In collaboration with:
Joseph Cina
Tao Wu

Abstract

Predictive modeling has become a pervasive human endeavor with the coupling of ever increasing computing capability in making predictions of real-world phenomena such as weather events, financial trends, outcomes in sports, etc. Harnessing the ability to create and develop models from data offers mankind the possibility of optimizing productivity under time constraints, enhance performance in competitive environments, or gain insights into phenomena that were once deemed unexplainable. In this paper, we will catch a glimpse of both the power and limitations of modeling in the world of machine learning through the enterprise of predicting sales price of the apartment market in Queens, NY.

1 Introduction

The essential building blocks of machine learning can be summarized in four words: data, algorithm, tuning, evaluation (Foss et. al. 2024). The *data* portion of machine learning concerns itself with the methods for creating training and test sets from raw (un)structured data. The *algorithm* refers to the machine learning procedure that will be implemented to be trained upon a given train set to test its predictive performance on test sets. The *tuning* refers to the configuration of an algorithm based upon its hyperparameters to best optimize both fitting and its generalization ("learning") of the data. Lastly, the *evaluation*, also known as *validation* phase in the machine learning paradigm deals with the measures to assess the quality of the predictions made by the algorithm upon new and previously unseen data. The project at hand is to go through each of these stages given a dataset on apartments in Queens to be modeled by three machine learning algorithms which are to be tuned to see how well each of them predicts the sales price of an apartment in Queens.

2 The Data

The data that was given at the onset of the project was a csv file on data regarding apartments in Queens, NY sold between February 2016 and February 2017. The dataset came from a raw download from Amazon's MTurk system. The original number of rows of the data set was $n = 2230$ and the original number of columns, indicating our original set of predictors was $p_{raw} = 55$. The features were comprised of 36 character, 5 logical, and 14 numeric data types.

2.1 Featurization, Errors and Missingness

The data munging phase of the project heavily dealt with the `full_address_or_zip_code` column of the original data set. There was a small finite number of entries for this column feature where the addresses were entered with one too many space characters and incomplete addresses. These entries were found by the help of the internet and were manually corrected. This was a crucial step in this machine learning pipeline because the key information in this column that would prove to be helpful in joining an additional table of pertinent data was the `zip_code`. So after the addresses were manually corrected the zip code for each address was extracted and a new column of zip codes took precedence over full address. Other columns such as `cats_allowed`, `dogs_allowed`, `dining_room_type`, `kitchen_type`, `garage_exists` were manually altered as well for the entries that either had incomplete words or misspellings.

The first 28 columns of the original dataset was dropped due to its irrelevance to the sale price of an apartment. The columns of bathrooms were made into a single column. The listing price which were numbers in the thousands were converted into the thousands. A new feature of age of the apartment was created. For the number of community districts that appeared less than 5 times in the data set were grouped under a level of "Other." The levels of dining, dining area, none were collapsed for the dining room column.

A table with data from `datacommons.techsoup.org` that incorporated the median age, the median income, the population and the housing units with respect to zip codes in NYC was left joined with the original dataset of apartment data. After joining the two tables together we proceeded with creating a missingness table and binding it to the joined table column-wise and

ran `MissForest` on the whole table. This became the design matrix for the rest of the project.

3 Modeling

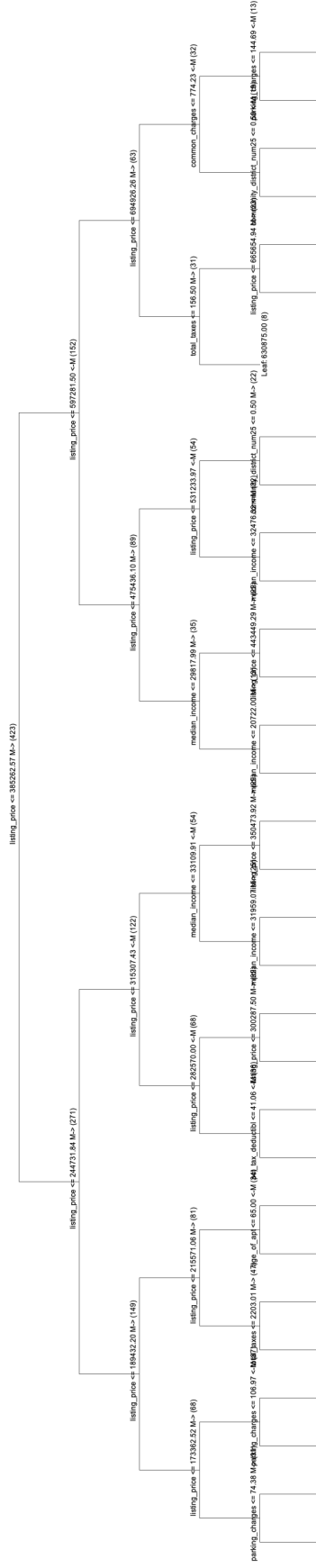
The main goal is to find a model that best captures the underlying relationship between the predictors from our `housing_design_matrix` and our target value `sale_price` and so we will compare three models, namely regression tree, linear regression, and random forest to see which one is most suitable for the task of predicting a Queens apartment's selling price.

3.1 Regression Tree Modeling

A regression tree was fit on our `housing_design_matrix` dataset and an illustration of the first few layers of our tree is found below. Based upon our regression tree here are the top 10 most important features needed to predict `sale_price`:

- `listing_price`
- `common_charges`
- `total_taxes`
- `median_income`
- `parking_charges`
- `pct_tax_deductible`
- `community_district_num25`
- `sq_footage`
- `median_income`
- `age_of_apt`

Seven out of the 10 features identified by the regression tree as being important for prediction on `sale_price` has to deal with money as either a charge, taxes, or income. Other features such as the `age_of_apt`, `community_district_num25`, `sq_footage` have more ancillary significance in the prediction of the sale price. We will see how some of these features most commonly seen in the first few layers of our regression tree will coincide with the most significant features found in OLS.



3.2 Linear Modeling

We fit a vanilla OLS onto our data and our performance metrics told us that 96% of the variation in the `sale_price` was explained by the independent variables in our model. Given our regression analysis the most significant features for prediction would be:

- `common_charges`
- `community_district_num25`
- `fuel_typegas`
- `fuel_typeoil`
- `maintenance_cost`
- `num_floors_in_building`
- `total_taxes`
- `listing_price`
- `is_missing_fuel_type`
- `is_missing_maintenance_cost`
- `is_missing_total_taxes`

From the regression tree modeling we saw that the regression tree's most important feature was the listing price and we see a similar result in the significance level from running OLS as seen by the summary output table (found at the end of the section). According to the coefficient for `listing_price` from our linear model, if all other features remain constant, an additional change of the listing price by one dollar will change the sale price of the apartment by \$1.07. There seems to be a direct correspondence between `listing_price` and `sale_price`. Given our results, the linear model will provide a robust prediction for our model.

Below is the table for the linear model `ols_housing_model`:

housing data imp train y			
<i>Predictors</i>	<i>Estimates</i>	<i>CI</i>	<i>p</i>
(Intercept)	-331686.36	-1104219.22 – 440846.50	0.399
cats allowedno	1546.48	-10943.31 – 14036.26	0.808
common charges	48.05	8.61 – 87.48	0.017
community district num25	-35951.11	-66045.48 – -5856.73	0.019
community district num26	-25633.16	-58323.00 – 7056.67	0.124
community district num27	-21990.68	-63148.86 – 19167.50	0.294
community district num28	-25678.62	-55313.24 – 3956.00	0.089
community district num29	-20957.40	-60608.40 – 18693.59	0.299
community district num30	-14264.98	-40789.38 – 12259.42	0.291
community district numOther	-13418.40	-45253.54 – 18416.73	0.408
coop condocondo	30409.84	-10494.56 – 71314.23	0.145
dining room typeformal	-4461.76	-15140.92 – 6217.40	0.412
dining room typeother	-12347.63	-26134.90 – 1439.64	0.079
dogs allowedyes	9234.29	-4822.19 – 23290.77	0.197
fuel typegas	28472.82	1897.75 – 55047.90	0.036

fuel typeoil	30390.75	2997.77 – 57783.74	0.030
fuel typeother	-14011.69	-55618.34 – 27594.97	0.508
kitchen typeeat-in	-2976.98	-15258.67 – 9304.72	0.634
kitchen typeefficiency	-1406.89	-13427.74 – 10613.95	0.818
maintenance cost	35.41	15.03 – 55.80	0.001
num bedrooms	-5934.37	-16228.32 – 4359.57	0.258
num floors in building	-1463.13	-2429.31 – -496.95	0.003
num total rooms	-4808.38	-11415.79 – 1799.03	0.153
parking charges	-56.30	-179.23 – 66.63	0.368
pct tax deductibl	104.28	-1085.96 – 1294.53	0.863
sq footage	-4.81	-18.64 – 9.02	0.494
total taxes	-5.19	-10.25 – -0.14	0.044
walk score	-11.27	-472.12 – 449.57	0.962
zip code11005	-645411.20	-1305741.28 – 14918.88	0.055
zip code11354	-18563.85	-296129.98 – 259002.28	0.895
zip code11355	9413.49	-370957.05 – 389784.04	0.961
zip code11357	23508.03	-134300.20 – 181316.26	0.770
zip code11360	-15421.49	-111400.03 – 80557.05	0.752
zip code11362	-47753.33	-127847.60 – 32340.94	0.242
zip code11364	17862.33	-109397.65 – 145122.31	0.783
zip code11367	138384.35	-35151.10 – 311919.80	0.118
zip code11368	248015.99	-219827.89 – 715859.87	0.298
zip code11372	89274.63	-171639.75 – 350189.01	0.501
zip code11373	152626.75	-259899.37 – 565152.87	0.467
zip code11374	26362.19	-160501.30 – 213225.68	0.782
zip code11375	224804.73	-198324.19 – 647933.65	0.297
zip code11377	183291.87	-204344.82 – 570928.56	0.353
zip code11414	-71387.66	-185989.81 – 43214.49	0.221
zip code11415	86980.90	-34662.60 – 208624.40	0.161
zip code11427	-10.36	-98016.73 – 97996.02	1.000
zip code11432	142358.44	-102777.72 – 387494.59	0.254
zip code11435	145673.89	-77125.84 – 368473.61	0.199
zip codeOther	74217.66	-136096.25 – 284531.57	0.488
num total bathroom	-9381.48	-23868.72 – 5105.76	0.204
listing price	1.07	1.00 – 1.14	<0.001

age of apt	-289.11	-639.09 – 60.86	0.105
median age	14044.77	-2516.77 – 30606.32	0.096
median income	-7.02	-18.66 – 4.61	0.236
population	-2.51	-10.06 – 5.04	0.514
housing units	-1.06	-16.93 – 14.80	0.895
is missing common charges	-11585.89	-35365.60 – 12193.82	0.339
is missing community district num	46541.66	-39396.90 – 132480.21	0.288
is missing dining room type	5000.87	-4432.68 – 14434.41	0.298
is missing fuel type	19130.68	1664.67 – 36596.70	0.032
is missing kitchen type	-11068.91	-43678.97 – 21541.15	0.505
is missing maintenance cost	-26201.50	-48983.60 – -3419.40	0.024
is missing num floors in building	465.14	-9857.03 – 10787.32	0.929
is missing parking charges	-7308.66	-17388.54 – 2771.21	0.155
is missing pct tax deductibl	3747.96	-6837.06 – 14332.98	0.487
is missing sq footage	4501.44	-4391.16 – 13394.04	0.320
is missing total taxes	36127.83	2045.97 – 70209.70	0.038
is missing age of apt	11837.40	-22421.31 – 46096.11	0.497
Observations	423		
R ² / R ² adjusted	0.965 / 0.958		

3.3 Random Forest Modeling

The random forest is expected to be the best choice out of the three supervised machine learning algorithms for prediction on new, previously unseen data because a random forest theoretically shrinks mean square error (MSE) the most out of the three. The reason why RF reduces MSE more than OLS and regressions trees is because in the bias-variance decomposition of MSE we have the expression

$$\text{MSE} = \mathbb{E} [\text{Bias}[g(\mathbf{x}_*)]^2] + \rho \mathbb{E} [\text{Var} [g_1(\mathbf{x}_*)]] + \sigma^2$$

and ρ decreases since random forest decorrelates the tree models, i.e. ρ decreases, by its inherent implementation of choosing a random subset of the p_{raw} features of size m_{try} , its hyperparameter. Employing this non-parametric model trades an increase for predictive accuracy for a loss in interpretability. Random forests do have a tendency to overfit the data but the overall generalization error does not increase due to an increase in the number of trees. The best thing that could be done was to tune the hyperparameters of 1) number of trees, 2) m_{try} , and 3) nodesize N_0 .

4 Performance Results

Error Metrics	Goodness-of-fit Metrics		
	ML Models		
	OLS	Regression Trees	Random Forest
in-sample R^2	0.9647099	0.9763553	0.9762317
oos R^2	0.9176330	0.9384443	0.9450022
in-sample RMSE	37700.9179909	28310.4669284	28384.3731015
oos RMSE	44318.3933056	37945.9415271	35867.7404608
k-fold oos RMSE	246627.5654708	241489.0188017	241069.0156759

Figure 1: A table of the performance metrics.

All of the in-sample and out-of-sample (oos) performance metrics for all three models have been tabulated in figure 1 above. The coefficient of determination, R^2 , for all three models, dipped in predictive performance as expected. With respect to R^2 predictive power OLS was least, followed by regression trees, and with random forest performing best out of the three ML models. Out of the three regression ML algorithms, it is expected that OLS would have fare least in the R^2 performance metric. OLS misspecification error and given the complexity of the true target function f would perform least in “fitting” the data as noted by its numerical R^2 metric. Regression tree and random forest perform nearly the same with respect to the in-sample R^2 metric but the random forest does slightly better out-of-sample. Random forest tend to out-perform the regression trees in out-of-sample prediction as noted by the R^2 metric because regression trees tend to overfit the training data which leads to lessen the ability to generalize prediction to newly unseen data whereas random forests employ bagging/ensemble techniques and randomizes the choices of features to train upon further diversifying and decorrelating the trees which enhances its predictive power out-of-sample.

With regards to the in-sample root mean square error (rmse), we see that OLS performs significantly worse than the regression trees and random forest while regression trees and random forest both perform comparatively. But the out-of-sample rmse for random forest is the least thereby confirming once more that the random forest model accurately predicts best out-of-sample out of the three models. Based upon its out-of-sample rmse, the random forest we fit onto to the dataset will on average have a difference between the predicted and actual values of the sale price will be roughly \$36,000 dollars compared with that of regression trees and OLS which are roughly \$38,000 and \$44,300, respectively.

To validate that our oos estimates were accurately indicative of our three models predictive accuracy, a 5-fold cross-validation was performed for each model with respect to its oos rmse and so we have some level of confidence that by in large on average the random forest model will have least oos rmse which crowns the random forest as the best model to predict the sales price of an apartment out of the three.

5 Discussion

Certain decisions and steps that were done in this project to evaluate the Queens apartment dataset and predict the target value of the sale price, does have me considering things I could have done differently or explored further if time, resources, and knowledge of machine learning were not limiting factors. I would have liked to have used the `model_type` feature for its suspected relationship to the price of an apartment. The decision to drop this feature regrettably came from its “messiness” and highly variable entry values that it contained. Also, the `listing_price` and `sale_price` features initially appear to have valuable information about each other, however, this was a moot point since none of the rows of the original dataset contained both of these values. We were relegated, instead, to use `MissForest` for the imputation of the missing values. Lastly, a feature like `pct_tax_deductible` appears to be quite informative regarding the types of potential buyers that would be interested in purchasing the property. There was a dataset I found online directly from the IRS that contained a wealth of tax information per zip code but the data had a complex structure that I could not, at the time, sift through or take advantage of in joining it to the original dataset. This is when I realized how useful the coupling of domain knowledge of a phenomena such as tax information and purchasing power along with the technical skills in manipulating datasets could have really benefited in gleaning extra information on the underlying relationship between the features of the Queens apartment market and the accuracy of a trained model to predict the value of a Queens’ apartment sale price.

I do not think my model is production ready nor do I deem my model viable threat to Zillow’s current performance. I suspect that both the size of the data set that the machine learning models were trained upon and the quality of the predictors that were both included and expanded upon limit the usefulness of the model. There are other viable machine learning algorithms that could have been employed such as boosting that could give a more predictive accuracy, as well as the possibility of employing other featurizations of the dataset that could have improved predictive performance of the model.

Acknowledgments

I would like to thank my two collaborators with whom I touched base with and consulted on how to start this project and see it through. I give my highest kudos to Tao for his unrivaled dedication to academic proficiency and for his commitment to not leaving any stone unturned regarding the details of the material that we have learned under Professor Kapelner over the last two semesters. And to Joe, a man with the seasoned wisdom and the wide-range experience of an elder statesmen, I give my utmost respect for the high bar of excellence he would set for us all. It was a pleasure and honor to share the classroom with these two distinguished gentlemen. I sincerely wish them both well in their future endeavors.

References

Foss N, Kotthoff L. (2024). Data and Basic Modeling. In Bischl B, Sonabend R, Kotthoff L, Lang M, (Eds.), Applied Machine Learning Using mlr3 in R. CRC Press. https://mlr3book.mlr-org.com/data_and_basic_modeling.html.

Code Appendix

```
1 ---
2 title: "Final Project for Math 642"
3 author: "Bryan Nevarez"
4 date: "Sunday, May 26th, 2024"
5 output: pdf_document
6 ---
7
8 # Packages/Libraries Setup
9 ```{r, set.seed(342)}
10 # remove everything from the current workspace
11 rm(list = ls())
12 #Set cache for seed
13 knitr::opts_chunk$set(cache = T)
14 #Memory allocation for Java ~10gb and Garbage Collection
15 options(java.parameters = c("-XX:+UseConcMarkSweepGC", "-Xmx10000m"))
16 #Packages to load
17 pacman::p_load(
18   ggplot2,
19   tidyverse,
20   data.table,
21   R.utils,
22   magrittr,
23   dplyr,
24   testthat,
25   YARF,
26   lubridate,
27   missForest,
28   parallel,
29   doParallel,
30   caret,
31   glmnet,
32   mlr,
33   mlr3
34 )
35 #Set CPU cores for YARF
36 num_of_cores = 8
37 set_YARF_num_cores(num_of_cores)
38 #Initialize rJava
39 library(rJava)
40 gc()
41 .jinit()
42 ```
43
44 # Data Import
45 ```{r}
46 library(readr)
47 #Read in the given raw housing data file
48 raw_housing_data = read_csv("housing_data_2016_2017.csv")
49
50 # Read in the nyc data per zip code found from the internet
51 nyc_data_per_zipcode = read_csv("New York City_CensusZipCodeTabulationArea.
   csv")
52
```

```

53 # checks the specifications of the columns for both raw data sets
54 spec(raw_housing_data)
55 spec(nyc_data_per_zipcode)
56
57 # View information on both raw data sets
58 skimr::skim(raw_housing_data)
59 skimr::skim(nyc_data_per_zipcode)
60 '''
61
62 # Data Preparation, Pt. I: raw_housing_data
63 '''{r}
64 # make raw_housing_data into a tibble
65 housing_data_tbl = as_tibble(raw_housing_data)
66
67 # drop the first 28 columns of housing_data_tbl
68 housing_data_tbl %<>%
69   select(-c(1:28))
70
71 # drop the model_type column (and the url column) due to it collinearity
72   with other columns
73 housing_data_tbl %<>%
74   select(-model_type, -url)
75
76 # inside view to housing_data_tbl
77 skimr::skim(housing_data_tbl)
78
79 # remove raw_housing_data
80 rm(raw_housing_data)
81
82 # Data Preparation, Pt. II: nyc_data_per_zipcode
83 '''{r}
84 # Make nyc zip code dataset into a tibble
85 nyc_data_per_zipcode_tbl = as_tibble(nyc_data_per_zipcode)
86
87 # deleting all rows deemed to be irrelevant or uninformative
88 nyc_data_per_zipcode_tbl %<>%
89   select(-c('placeDcid', 'Date:Median_Age_Person', 'Source:Median_Age_Person',
90             'Date:Median_Income_Person', 'Source:Median_Income_Person', 'Date:Count_
91             Person', 'Source:Count_Person', 'Date:Count_HousingUnit', 'Source:Count_
92             HousingUnit', 'Date:Monthly_Median_GrossRent_HousingUnit', 'Value:Monthly_
93             Median_GrossRent_HousingUnit', 'Source:Monthly_Median_GrossRent_
94             HousingUnit'))
95
96 # renaming the columns
97 nyc_data_per_zipcode_tbl %<>%
98   rename(zip_code = placeName, median_age = 'Value:Median_Age_Person',
99          median_income = 'Value:Median_Income_Person', population = 'Value:Count_
100          Person', housing_units = 'Value:Count_HousingUnit')
101
102 # inside view of nyc_data_per_zipcode_tbl
103 skimr::skim(nyc_data_per_zipcode_tbl)
104
105 # remove nyc_data_per_zipcode
106 rm(nyc_data_per_zipcode)

```

```

100 '''
101
102 # Data Cleaning I (Errors, Duplicates, Outliers Handling)
103 '''{r}
104 # remove the dollar symbol from each of the entries in the relevant columns
105 #housing_data_tbl$common_charges = gsub("\\$", "", housing_data_tbl$common_
    charges)
106 #housing_data_tbl$maintenance_cost = gsub("\\$", "", housing_data_tbl$
    maintenance_cost)
107 #housing_data_tbl$parking_charges = gsub("\\$", "", housing_data_tbl$parking
    _charges)
108 #housing_data_tbl$total_taxes = gsub("\\$", "", housing_data_tbl$total_taxes
    )
109 #housing_data_tbl$sale_price = gsub("\\$", "", housing_data_tbl$sale_price)
110 #housing_data_tbl$sale_price = gsub("\\,", "", housing_data_tbl$sale_price)
111 #housing_data_tbl$listing_price_to_nearest_1000 = gsub("\\$", "", #housing_
    data_tbl$listing_price_to_nearest_1000)
112
113 # one line piping command to get rid of dollar sign and commas from the
    above commented cols
114 housing_data_tbl %<>%
115   mutate(across(c(3,13,19,21,23,25), gsub, pattern = "\\$|,", replacement =
    ""))
116
117 # cast various features into appropriate data type
118 housing_data_tbl %<>%
119   mutate(common_charges = as.numeric(common_charges),
120          maintenance_cost = as.numeric(maintenance_cost),
121          parking_charges = as.numeric(parking_charges),
122          total_taxes = as.numeric(total_taxes),
123          listing_price_to_nearest_1000 = as.numeric(listing_price_to_nearest
    _1000),
124          sale_price = as.numeric(sale_price),
125          approx_year_built = as.integer(housing_data_tbl$approx_year_built))
126
127
128 # clear out human errors in character-type features
129
130 # str_extract() extracts the first occurrence of a pattern matching a five-
    digit zip code (\\b\\d{5}\\b) from each element of the full_address_or_
    zip_code column.
131 # The result is stored in a new column called zip_code in housing_data_tbl.
132 housing_data_tbl = housing_data_tbl %<>%
133   mutate(zip_code = sub(".*\\b(\\d{5})\\b.*", "\\1", housing_data_tbl$full_
    address_or_zip_code))
134
135 # drop full_address_or_zip_code column and date of sale after extraction
    operation
136 housing_data_tbl %<>%
137   select(-full_address_or_zip_code, -date_of_sale )
138
139 # the reasonably small number of misspellings, inaccuracies in the certain
    data entries
140 # have been carefully corrected and standardized one-by-one in the code
    below

```

```

141
142 # Zip code Column
143 housing_data_tbl %<>%
144   mutate(zip_code = replace(zip_code, zip_code == "138-35 Elder Ave,
145     Flushing NY, 1135", "11355")) %>%
146   mutate(zip_code = replace(zip_code, zip_code == "32-42 89th St, E.
147     Elmhurst NY, 1136", "11369")) %>%
148   mutate(zip_code = replace(zip_code, zip_code == "34-30 78th St, Jackson
149     Heights NY, 1137", "11375")) %>%
150   mutate(zip_code = replace(zip_code, zip_code == "35-25 77 St, Jackson
151     Heights NY, 1137", "11375")) %>%
152   mutate(zip_code = replace(zip_code, zip_code == "61-20 Grand Central Pky,
153     Forest Hills NY, 11375", "11355")) %>%
154   mutate(zip_code = replace(zip_code, zip_code == "78-07 Springfield Blvd,
155     Bayside NY, 1136", "11364")) %>%
156   mutate(zip_code = replace(zip_code, zip_code == "80-35 Springfield Blvd,
157     Queens Village NY, 11429", "11355")) %>%
158   mutate(zip_code = replace(zip_code, zip_code == "138-35 Elder Ave,
159     Flushing NY, 1135", "11355")) %>%
160   mutate(zip_code = replace(zip_code, zip_code == "32-42 89th St, E.
161     Elmhurst NY, 1136", "11369")) %>%
162   mutate(zip_code = replace(zip_code, zip_code == "35-25 77 St, Jackson
163     Heights NY, 1137", "11372")) %>%
164   mutate(zip_code = replace(zip_code, zip_code == "61-20 Grand Central Pky,
165     Forest Hills NY, 11375", "11355")) %>%
166   mutate(zip_code = replace(zip_code, zip_code == "78-07 Springfield Blvd,
167     Bayside NY, 1136", "11364")) %>%
168   mutate(zip_code = replace(zip_code, zip_code == "80-35 Springfield Blvd,
169     Queens Village NY, 11429", "11355")) %>%
170   mutate(zip_code = replace(zip_code, zip_code == "61-20 Grand Central Pky,
171     Forest Hills NY, 1137", "11375")) %>%
172   mutate(zip_code = replace(zip_code, zip_code == "80-35 Springfield Blvd,
173     Queens Village NY, 1142", "11429")) %>%
174   mutate(zip_code = replace(zip_code, zip_code == "61-20 Grand Central Pky,
175     Forest Hills NY, 1137", "11375")) %>%
176   mutate(zip_code = replace(zip_code, zip_code == "80-35 Springfield Blvd,
177     Queens Village NY, 1142", "11429"))
178
179 # cats_allowed column
180 housing_data_tbl %<>%
181   mutate(cats_allowed = replace(cats_allowed, cats_allowed == "y", "yes"))
182
183 # dogs_allowed column
184 housing_data_tbl %<>%
185   mutate(dogs_allowed = replace(dogs_allowed, dogs_allowed == "yes89", "yes"
186     ))
187
188 # dining_room_type column
189 housing_data_tbl %<>%
190   mutate(dining_room_type = replace(dining_room_type, dining_room_type == "
191     yes89", "yes"))
192
193 # fuel_type column
194 housing_data_tbl %<>%
195   mutate(fuel_type = replace(fuel_type, fuel_type == "Other", "other")) %>%

```

```

177 mutate(fuel_type = replace(fuel_type, fuel_type == "none", NA))
178
179 # kitchen_type column
180 housing_data_tbl %<>%
181   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "eat in", "eat
    -in")) %>%
182   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "Eat In", "eat
    -in")) %>%
183   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "Eat in", "eat
    -in")) %>%
184   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "eatin", "eat-
    in")) %>%
185   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "efficiency
    kitchen", "efficiency")) %>%
186   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "efficiency
    kitchen", "efficiency")) %>%
187   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "efficiemcy",
    "efficiency")) %>%
188   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "efficiency
    ktchen", "efficiency")) %>%
189   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "Combo", "
    combo")) %>%
190   mutate(kitchen_type = replace(kitchen_type, kitchen_type == "1955", NA))
191
192 # garage_exists column
193 housing_data_tbl %<>%
194   mutate(garage_exists = replace(garage_exists, garage_exists == "
    Underground", "underground")) %>%
195   mutate(garage_exists = replace(garage_exists, garage_exists == "UG", "
    underground")) %>%
196   mutate(garage_exists = replace(garage_exists, garage_exists == "Yes", "yes
    ")) %>%
197   mutate(garage_exists = replace(garage_exists, garage_exists == "eys", "yes
    ")) %>%
198   mutate(garage_exists = replace(garage_exists, garage_exists == "1", NA))
199
200 # view cleaned housing_data_tbl
201 skimr::skim(housing_data_tbl)
202
203 # Data Manipulation (Featurization)
204 ```{r}
205 # combine num_full_bathrooms and num_half_bathrooms for a new variable of
    num_bathroom
206 housing_data_tbl %<>%
207   mutate(num_half_bathrooms = replace(num_half_bathrooms, is.na(num_half_
    bathrooms), 0)) %>%
208   mutate(num_total_bathroom = num_full_bathrooms + (num_half_bathrooms / 2))
209
210 # drop columns of num_full_baths and num_half_bathrooms
211 housing_data_tbl %<>%
212   select(-num_full_bathrooms, -num_half_bathrooms)
213
214 # multiply listing_price_to_nearest_1000 by 1000 so that sale price share
    #the same unit of dollar
215
216 housing_data_tbl %<>%

```

```

217 mutate(listing_price = listing_price_to_nearest_1000 * 1000) %>%
218 select(-listing_price_to_nearest_1000)
219
220 # age of apartment
221 housing_data_tbl$age_of_aprt = 2017 - housing_data_tbl$approx_year_built
222
223 # dropped approx_year_built and garage_exists
224 housing_data_tbl %<>%
225   select(-approx_year_built, -garage_exists)
226
227 # combining community district values where there are less than 5 counts and
    labeling them other
228 community_district_num_counts = table(housing_data_tbl$community_district_
    num)
229 rare_levels = names(community_district_num_counts[community_district_num_
    counts < 20])
230 housing_data_tbl$community_district_num = as.character(housing_data_tbl$
    community_district_num)
231 housing_data_tbl$community_district_num[housing_data_tbl$community_district_
    num
232                                     %in% rare_levels] = "Other"
233 housing_data_tbl$community_district_num = as.factor(housing_data_tbl$
    community_district_num)
234
235 # combining zip codes where there are less than 5 counts and labeling them
    other
236 zip_code_counts = table(housing_data_tbl$zip_code)
237 rare_levels = names(zip_code_counts[zip_code_counts < 25])
238 housing_data_tbl$zip_code = as.character(housing_data_tbl$zip_code)
239 housing_data_tbl$zip_code[housing_data_tbl$zip_code %in% rare_levels] = "
    Other"
240 housing_data_tbl$zip_code = as.factor(housing_data_tbl$zip_code)
241
242 # collapsing levels of dining, dining area, none in dining room type
243 housing_data_tbl %<>%
244   mutate(dining_room_type = replace(dining_room_type,
245                                     dining_room_type == "dining", "other"))
246   %>%
247   mutate(dining_room_type = replace(dining_room_type,
248                                     dining_room_type == "dining area",
249                                     "other")) %>%
250   mutate(dining_room_type = replace(dining_room_type,
251                                     dining_room_type == "none", "other"))
252 housing_data_tbl$dining_room_type = as.factor(housing_data_tbl$dining_room_
    type)
253
254 # view housing_data_tbl
255 skimr::skim(housing_data_tbl)
256 ‘‘‘
257
258
259 # Joining Tables
260 ‘‘‘{r}

```

```

261 #left join nyc_data_per_zipcode_tbl to housing data table before casting
262 #zip_code feature as a factor type
263 nyc_data_per_zipcode_tbl$zip_code = as.character(nyc_data_per_zipcode_tbl$
      zip_code)
264 housing_data_tbl = left_join(housing_data_tbl, nyc_data_per_zipcode_tbl,
265                               by = join_by("zip_code" == "zip_code"))
266
267 # factorize character-type features
268 housing_data_tbl %<>%
269   mutate(across(c(cats_allowed, dogs_allowed, coop_condo, fuel_type,
270                   kitchen_type, zip_code), as.factor))
271
272 # view housing_data_tbl
273 skimr::skim(housing_data_tbl)
274 ```
275
276 # Data Cleaning II (Missingness)
277 ```{r}
278 # constructing a missingness table with 20 columns that represents 20
      columns
279 #with complete rate less than 1 in the original tibble table
280 M = as_tibble(apply(is.na(housing_data_tbl), 2, as.numeric))
281 colnames(M) = paste("is_missing_", colnames(housing_data_tbl), sep = "")
282 M %<>%
283   select_if(function(x){sum(x) > 0})
284
285 # create the design matrix by cbinding the M table - missingness itself is
286 #informative. We want to preserve it in the design matrix
287 housing_data_miss = cbind(housing_data_tbl, M)
288
289 # smartly impute dataset using "missForest"
290 housing_data_imp = missForest(data.frame(housing_data_miss))$ximp
291
292 # dummify all factor variables and return a design matrix ready to be split
293 housing_data_imp = model.matrix(~ 0 + ., housing_data_imp)
294 housing_data_imp = as.data.frame(housing_data_imp)
295 skimr::skim(housing_data_imp)
296 ```
297
298 # Design Matrix & Test Split
299 ```{r}
300 set.seed(1717)
301 K = 5
302 # we will now work only on those rows that have an actual sale_price for
      training and testing
303 # table(housing_data_imp$is_missing_sale_price) to see for ourselves that "
      is_missing_sale_price == 0"
304 # represents 528 rows that have the actual real world values in the outcome
      variable columns
305 housing_design_matrix = housing_data_imp[housing_data_imp$is_missing_sale_
      price == 0, ]
306 housing_design_matrix_y = housing_design_matrix$sale_price
307 housing_design_matrix = as_tibble(housing_design_matrix)
308 housing_design_matrix %<>%
309   select(-sale_price, -is_missing_sale_price)

```



```

310
311 # view housing_design_matrix
312 skimr::skim(housing_design_matrix)
313
314 # store indices of train set and test set
315 test_idx = sample(nrow(housing_design_matrix)/K, replace = FALSE)
316 train_idx = setdiff(1 : nrow(housing_design_matrix), test_idx)
317
318 # subsetting the training set from main model matrix
319 housing_data_imp_train_X = housing_design_matrix[train_idx, ]
320 housing_data_imp_train_y = housing_design_matrix_y[train_idx]
321
322 # subsetting the testing set from the main model matrix
323 housing_data_imp_test_X = housing_design_matrix[test_idx, ]
324 housing_data_imp_test_y = housing_design_matrix_y[test_idx]
325
326 housing_data_imp_train_X = as.data.frame(housing_data_imp_train_X)
327
328 # export housing data feature summary
329 housing_summary = skimr::skim(housing_data_imp)
330 housing_summary_df = as.data.frame(housing_summary)
331 write.csv(housing_summary_df, file = "housing_summary.csv", row.names =
332   FALSE)
333
334 # Linear Modeling
335 ```{r}
336 # vanilla OLS regression
337 ols_housing_model = lm(housing_data_imp_train_y ~ ., data = housing_data_imp
338   _train_X)
339
340 # in sample metrics (from scratch)
341 insample_ols_hats = ols_housing_model$fitted.values
342 insample_ols_residuals = housing_data_imp_train_y - insample_ols_hats
343 insample_ols_SSE = sum(insample_ols_residuals^2)
344 insample_ols_SST = sum((housing_data_imp_train_y - mean(housing_data_imp_
345   train_y))^2)
346 insample_ols_rsquared_from_scratch = 1 - insample_ols_SSE / insample_ols_SST
347   # R-squared (in-sample)
348
349 insample_ols_rmse_from_scratch = sqrt( insample_ols_SSE / (nrow(housing_data
350   _imp_train_X) - 2 ))
351
352 #in sample metrics (built-in R)
353 insample_ols_rsquared = summary(ols_housing_model)$r.squared # R-squared (in
354   -sample)
355 insample_ols_RMSE = summary(ols_housing_model)$sigma # rmse (in-sample)
356
357 # predict on test set
358 ols_y_hat = predict(ols_housing_model, newdata = housing_data_imp_test_X)
359
360 #out of sample metrics
361 oos_ols_residuals = housing_data_imp_test_y - ols_y_hat
362 SSE = sum(oos_ols_residuals^2)
363 SST = sum((housing_data_imp_test_y - mean(housing_data_imp_test_y))^2)

```

```

359
360 # R-squared
361 oos_ols_rsquared = 1 - SSE / SST
362
363 # oos RMSE
364 oos_ols_RMSE = sqrt(SSE / (nrow(housing_data_imp_test_X) - 2))
365
366 # show results
367 PercentDiff = (ols_y_hat - housing_data_imp_test_y) / housing_data_imp_test_
    y
368 PredictSummary = cbind(ols_y_hat, housing_data_imp_test_y, PercentDiff)
369 write.csv(PredictSummary, file = "PredictSummary.csv", row.names = FALSE)
370 ""
371
372 # Regression Trees
373 ""{r}
374 # locate the optimal node size hyperparameter for the regression tree model
375 max_nodesize = 70
376 in_sample_errors = array(NA, max_nodesize)
377 oos_errors = array(NA, max_nodesize)
378
379 for (i in 1 : max_nodesize){
380     regression_tree_mode = YARFCART(data.frame(x = housing_data_imp_train_X),
    housing_data_imp_train_y, nodesize = i, calculate_oob_error = FALSE)
381     y_hat = predict(regression_tree_mode,
382                     data.frame(x = housing_data_imp_train_X))
383     in_sample_errors[i] = sd(housing_data_imp_train_y - y_hat)
384     y_hat_test = predict(regression_tree_mode,
385                          data.frame(x = housing_data_imp_test_X))
386     oos_errors[i] = sd(housing_data_imp_test_y - y_hat_test)
387 }
388
389 # visualize hyperparameter tuning
390 ggplot(
391     rbind(
392         data.frame(s_e = in_sample_errors,
393                   num_nodesize = 1 : max_nodesize, series = "in-sample"),
394         data.frame(s_e = oos_errors,
395                   num_nodesize = 1 : max_nodesize, series = "out-of-sample")
396     )) +
397     geom_line(aes(x = num_nodesize, y = s_e, col = series))
398
399 # optimal nodesize
400 optimal_nodesize = which.min(oos_errors)
401
402 # fit regression tree model using the optimal nodesize
403 tree_model = YARFCART(housing_data_imp_train_X, housing_data_imp_train_y,
404                       nodesize = optimal_nodesize, calculate_oob_error =
    TRUE,
405                       verbose = FALSE)
406 illustrate_trees(tree_model, max_depth = 6, open_file = TRUE)
407
408 # yhats from regression tree model
409 tree_model_train_yHats = predict(tree_model, housing_data_imp_train_X)
410 tree_model_yHats = predict(tree_model, housing_data_imp_test_X)

```

```

411
412
413 # in sample R-squared
414 insample_regtree_residuals = housing_data_imp_train_y - tree_model_train_
    yHats
415 insample_regtree_SSE = sum(insample_regtree_residuals^2)
416 insample_regtree_SST = sum((housing_data_imp_train_y - mean(housing_data_imp
    _train_y))^2)
417 insample_regtree_rsquared = 1 - insample_regtree_SSE / insample_regtree_SST
418
419 # oos sample R-squared
420 oos_regtree_residuals = housing_data_imp_test_y - tree_model_yHats
421 oos_regtree_SSE = sum(oos_regtree_residuals^2)
422 oos_regtree_SST = sum((housing_data_imp_test_y - mean(housing_data_imp_test_
    y))^2)
423 oos_regtree_rsquared = 1 - oos_regtree_SSE / oos_regtree_SST
424
425 # RMSE
426 insample_rmse_regtree = sqrt(sum((housing_data_imp_train_y-tree_model_train_
    yHats)^2)/length(housing_data_imp_train_y)) # insample rmse
427
428 oos_rmse_regtree = sqrt(sum((housing_data_imp_test_y-tree_model_yHats)^2)/
    length(housing_data_imp_test_y)) # oos rmse
429
430 # show results
431 PercentDiffTree = (tree_model_train_yHats - housing_data_imp_test_y) /
    housing_data_imp_test_y
432 PredictSummaryTree = cbind(tree_model_train_yHats, housing_data_imp_test_y,
    PercentDiffTree)
433 write.csv(PredictSummaryTree, file = "PredictSummaryTree.csv", row.names =
    FALSE)
434 '''
435
436 # Random Forest
437 '''{r}
438 # find the best tuning parameters, starting with num_tree
439 num_trees_values = c(1, 2, 5, 10, 20, 30, 40, 50, 100, 200, 300, 400, 500)
440 oob_se_rf_mod_by_num_trees = array(NA, length(num_trees_values))
441 for (m in 1 : length(num_trees_values)){
442     mod_rf = YARF(housing_design_matrix, housing_design_matrix_y,
443                 nodesize = round(.1 * nrow(housing_data_imp_train_X)),
444                 num_trees = num_trees_values[m], mtry = 60)
445     oob_se_rf_mod_by_num_trees[m] = mod_rf$rmse_oob
446 }
447 # visualize hyperparameter tuning results
448 ggplot(data.frame(num_trees = num_trees_values,
449                 oob_se = oob_se_rf_mod_by_num_trees)) +
450     aes(x = num_trees, y = oob_se) +
451     geom_point() + geom_line() + scale_x_log10()
452
453 # choose optimal number of trees
454 optimal_num_trees = which.min(oob_se_rf_mod_by_num_trees)
455
456 # NEXT, find the best M_try parameter using the optimal num_trees
457 oob_se_by_mtry = array(NA, ncol(housing_data_imp_train_X))

```

```

458 for(m in 1: ncol(housing_data_imp_train_X)){
459   mod_rf = YARF(housing_design_matrix, housing_design_matrix_y,
460                 nodesize = round(.1 * nrow(housing_data_imp_train_X)),
461                 num_trees = optimal_num_trees, mtry = m)
462   oob_se_by_mtry[m] = mod_rf$rmse_oob
463 }
464
465 # visualize hyperparameter tuning results
466 ggplot(data.frame(num_mtry = seq(1, ncol(housing_data_imp_train_X)),
467                  oob_se = oob_se_by_mtry)) +
468   aes(x = num_mtry, y = oob_se) +
469   geom_point() + geom_line()
470
471 # choose optimal mtry parameter
472 optimal_num_mtry = which.min(oob_se_by_mtry)
473
474 # locate the optimal node size hyperparameter for the regression tree model
475 max_nodesize = 70
476 oos_errors = array(NA, max_nodesize)
477 for (i in 1 : max_nodesize){
478   rf_mod = YARF(housing_design_matrix, housing_design_matrix_y, nodesize = i
479                 , num_trees = optimal_num_trees, mtry = optimal_num_mtry)
480   y_hat = predict(rf_mod, housing_design_matrix)
481   oos_errors[i] = rf_mod$rmse_oob
482 }
483
484 # visualize hyperparameter tuning results
485 ggplot(
486   rbind(
487     data.frame(s_e = oos_errors,
488               num_nodesize = 1 : max_nodesize, series = "out-of-sample")
489   )) +
490   geom_line(aes(x = num_nodesize, y = s_e, col = series))
491
492 # choose optimal value for nodesize parameter
493 optimal_nodesize = which.min(oos_errors)
494
495 # let's fit a RandomForest to our important feature set
496 rf_model = YARF(housing_data_imp_train_X, housing_data_imp_train_y,
497                 nodesize = optimal_nodesize, num_trees = optimal_num_trees,
498                 mtry = optimal_num_mtry, verbose = FALSE)
499
500 # out-of-bag metric
501 oob_rf_rmse = rf_model$rmse_oob
502 illustrate_trees(rf_model, max_depth = 6, open_file = TRUE)
503
504 # yhats from random forest model
505 rf_model_train_yHats = predict(rf_model, housing_data_imp_train_X)
506 rf_model_test_yHats = predict(rf_model, housing_data_imp_test_X)
507
508 # in sample R-squared
509 insample_rf_residuals = housing_data_imp_train_y - rf_model_train_yHats
510 insample_rf_SSE = sum(insample_rf_residuals^2)

```

```

511 insample_rf_SST = sum((housing_data_imp_train_y - mean(housing_data_imp_
    train_y))^2)
512 insample_rf_rsquared = 1 - insample_rf_SSE / insample_rf_SST
513
514 # oos sample R-squared
515 oos_rf_residuals = housing_data_imp_test_y - rf_model_yHats
516 oos_rf_SSE = sum(oos_rf_residuals^2)
517 oos_rf_SST = sum((housing_data_imp_test_y - mean(housing_data_imp_test_y))
    ^2)
518 oos_rf_rsquared = 1 - oos_rf_SSE / oos_rf_SST
519
520 # RMSE
521 insample_rmse_rf = sqrt(sum((housing_data_imp_train_y-rf_model_train_yHats)
    ^2)/length(housing_data_imp_train_y)) # insample rmse
522
523 oos_rmse_rf = sqrt(sum((housing_data_imp_test_y-rf_model_yHats)^2)/length(
    housing_data_imp_test_y)) # oos rmse
524
525
526
527 # show results
528 print("percentage gain / loss in predictive performance")
529 PercentDiffTree = (tree_model_train_yHats - housing_data_imp_test_y)
530                     / housing_data_imp_test_y * 100
531 PredictSummaryTree = cbind(tree_model_train_yHats, housing_data_imp_test_y,
532                             PercentDiffTree)
533 write.csv(PredictSummaryTree, file = "PredictSummaryTree.csv", row.names =
    FALSE)
534 '''
535
536 # K-fold CV
537 '''{r}
538 set.seed(1984)
539 n_sub = nrow(housing_design_matrix)
540 temp = rnorm(n_sub)
541
542 # create K equally sized folds
543 folds_vec = cut(temp, breaks = quantile(temp, seq(0, 1, length.out = K + 1))
    ,
544               include.lowest = TRUE, labels = FALSE)
545 rm(temp)
546
547 # initialize vectors to store results
548 ols_rmse = numeric(K)
549 cart_rmse = numeric(K)
550 rf_rmse = numeric(K)
551
552 # define Root Mean Squared Error (RMSE) function
553 rmse = function(actual, predicted) {
554     sqrt(mean((actual - predicted)^2))
555 }
556
557 # perform k-fold cross-validation
558 for (i in 1:K) {
559     # split the data into training and testing sets

```

```

560 test_indices_k = which(folds_vec == K)
561 train_indices_k = which(folds_vec != K)
562
563 train_data_X = housing_design_matrix[train_indices_k, ]
564 train_data_y = housing_design_matrix_y[train_indices_k]
565
566 test_data_X = housing_design_matrix[test_indices_k, ]
567 test_data_y = housing_design_matrix_y[train_indices_k]
568
569 # OLS Model
570 ols_model = lm(train_data_y ~ ., data = train_data_X)
571 ols_predictions = predict(ols_model, test_data_X)
572 ols_rmse[i] = rmse(test_data_y, ols_predictions)
573
574 # Regression Tree Model
575 cart_model = YARFCART(train_data_X, train_data_y, nodesize = optimal_
    nodesize, calculate_oob_error = TRUE, verbose = FALSE)
576 cart_predictions = predict(cart_model, test_data_X)
577 cart_rmse[i] = rmse(test_data_y, cart_predictions)
578
579 # Random Forest Model
580 rf_model = YARF(train_data_X, train_data_y, nodesize = optimal_nodesize,
    num_trees = optimal_num_trees, mtry = optimal_num_mtry,
    verbose = FALSE)
581 rf_predictions = predict(rf_model, test_data_X)
582 rf_rmse[i] = rmse(test_data_y, rf_predictions)
583 }
584
585 # calculate the mean RMSE for each of the three models
586 mean_ols_rmse = mean(ols_rmse)
587 mean_cart_rmse = mean(cart_rmse)
588 mean_rf_rmse = mean(rf_rmse)
589
590 # print the results
591 cat("Mean out-of-sample RMSE for OLS:", mean_ols_rmse, "\n")
592 cat("Mean out-of-sample RMSE for Regression Tree:", mean_cart_rmse, "\n")
593 cat("Mean out-of-sample RMSE for Random Forest:", mean_rf_rmse, "\n")
594
595 # Performance Validation Summary
596
597 # prevent table of values to express numbers in scientific notation
598 options(scipen=9)
599
600 # create a dataframe to report goodness of fit metrics in one clear table
601 goodness_of_fit_metrics = data.frame(
602   error_metrics = c("in-sample R^2", "oos R^2", "in-sample RMSE", "oos RMSE"
603     ,
604     "k-fold oos RMSE"),
605   OLS = c(insample_ols_rsquared, oos_ols_rsquared, insample_ols_RMSE,
606     oos_ols_RMSE, mean_ols_rmse),
607   Trees = c(insample_regtree_rsquared, oos_regtree_rsquared,
608     insample_rmse_regtree, oos_rmse_regtree, mean_cart_rmse),

```

```

612 RF = c(insample_rf_rsquared, oos_rf_rsquared, insample_rmse_rf, oos_rmse_
613       rf,
614       mean_rf_rmse )
615 )
616 colnames(goodness_of_fit_metrics) = c("Error Metrics", "OLS",
617                                       "Regression Trees", "Random Forest")
618
619 # load gt package
620 library(gt)
621
622 # use gt package to create a visually appealing table
623 final_metrics_table_gt = gt(goodness_of_fit_metrics)
624
625 final_metrics_table_gt <-
626   final_metrics_table_gt |>
627   tab_header(
628     title = "Goodness-of-fit Metrics"
629   ) |>
630   tab_spanner(
631     label = "ML Models",
632     columns = c("OLS", "Regression Trees", "Random Forest")
633   ) |>
634   cols_align(
635     align = "center",
636     columns = everything()
637   )
638
639
640 # show the gt Goodness-of-Fit Metrics
641 final_metrics_table_gt
642
643 '''
644
645 # Shipping Final Model Trained on All Data
646 '''{r}
647 mod_star = YARF(data.frame(housing_data_imp_train_X, housing_data_imp_test_X
648                            ), data.frame(housing_data_imp_train_y, housing_data_imp_test_y),
649           nodesize = round(.1 * nrow(housing_data_imp_train_X)), num_trees = 1000,
650           mtry = 60, verbose = FALSE)
651
652 # ship g_final
653 g_final = function(x_star){
654   predict(mod_star, x_star)
655 }
656 '''

```